PROJECT REPORT

GREEN PROJECT

DONE

Nexus

Author(s):

Famura

Contents

1. Introduction 3
2. Project Technologies 3
   2.1. Modules 3
   2.2. Programming Languages 3
   2.3. Libraries and External Components 4
3. Project Architecture 4
   3.1. Transfer 4
      3.1.1. Kimlik Doğrulama ve Yetkilendirme 4
      3.1.2. Kaynak Yönetimi ve Versiyonlama 5
      3.1.3. Pull (Download) Flow 6
      3.1.4. Push (Upload) Flow 6
      3.1.5. Manage (Delete) 7
   3.2. Web Site 7
      3.2.1. Static Content Serving 7
      3.2.2. Interface 8
   3.3. Contact Form 9
4. Project Analysis 9
   4.1. Test 9
   4.2. Upgradeable Things 10
5. Project Comments 10
   5.1. Possible Use Cases 10
   5.2. To-Do's When Using 10
   5.3. Developer Comments 10
6. Credits 10
7. Kaynaklar 11
8. Disk İçeriği 11

1. Introduction

Nexus, is designed as a personal portfolio and secure file transfer system consisting of two main parts. The **Web Site** section provides an interface that presents projects, experiences, and reports to users with **Turkish and English language support**.

The **Transfer** section provides a service that performs **versioned file transfers to public or private resources** using signed and encrypted requests.

The project is built on **Spring Boot** and combines both **static web content** and **transfer API endpoints** in a single application.

The website can be accessed at famura.site or www.famura.site.

2. Project Technologies

2.1. Modules

- **Linux (Ubuntu Mate):**
  The project was developed and is currently used on my Ubuntu Mate machine.
  It has not been tested on other Ubuntu versions or Linux environments, but due to Java's portability it will most likely run without problems.
- **Windows (Windows 11):**
  Thanks to Java's platform independence, the project can also run on Windows.
  When I perform maintenance on my Linux machine, I sometimes run the system from my Windows computer.
  Other Windows versions have not been tested, but they are expected to work as well.

2.2. Programming Languages

- **Java(Java 17/Maven) – Intellij IDEA:**
  The entire project is written in Java. I chose **IntelliJ IDEA** because of its strong support for Git and Maven.
- **Spring Boot(Spring Boot 3.3.7) – Intellij IDEA:**
  Spring Boot was used to initialize and run the application.
  Although Spring Boot is technically not a programming language but a Java framework, it plays a major role in the architecture of the project and therefore deserves mention here. After previously working with plain Spring, switching to Spring Boot made development significantly easier.
- **SQLite(SQLite 3) – Linux Terminal:**
  SQLite is used for the transfer subsystem. Since the system is designed to run primarily on a single machine, using heavier solutions such as Docker or MySQL was unnecessary. SQLite proved to be fully sufficient for the project. All database management operations were performed through the Linux terminal.
- **HTML/CSS & Javascript– Linux Firefox & Intellij IDEA:**
  The user interface of the website was built using HTML, CSS, and JavaScript.
  This includes: interface design, scripts, routing logic, dictionary-based language switching, styling, reference displays. Development was done in IntelliJ IDEA and tested in the Firefox browser on Linux.

2.3. Libraries and External Components

- **Spring Boot Starter Web:** MVC framework and REST endpoints
- **Spring Data JPA:** ORM and repository layer.
- **Hibernate:** SQLite dialect and JPA integration
- **Spring Boot Starter Mail:** sending messages from the contact form.
- **ClamAV (clamscan):** antivirus scanning for uploaded files.
- **Cloudflared:** used to expose the locally running project to the internet through a tunnel

3. Project Architecture

The project is divided into two main components: Transfer & Web_Site.

3.1. Transfer

The transfer component is not accessible to normal website users.
It is designed to be accessed only by **Malzahar (the client application)** or any other client that communicates with the same endpoints.

However, the security of this system does not rely on secrecy but on **cryptographic algorithms**.

The system uses:

- **HMAC** for request signing
- **RSA** for secure payload encryption
- **AES** for bulk data encryption

Without the required cryptographic keys, it is computationally infeasible to:

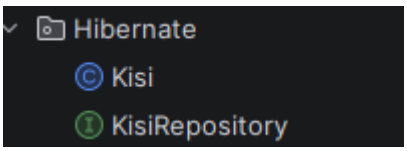
- access transferred data
- request data from the server
- send malicious requests to the server
- access user account information

```java
private String rsa_encrypt(String jsonPayload, String name) throws Exception {  1 usage  Famura71
    if (jsonPayload == null || jsonPayload.isBlank()) {
        throw new IllegalArgumentException("missing json payload");
    }
    if (name == null || name.isBlank()) {
        throw new IllegalArgumentException("missing name");
    }

    Optional<Kisi> kisi = kisiRepository.findByAd(name);
    if (kisi.isEmpty()) {
        throw new IllegalStateException("user not found");
    }

    String publicKeyBase64 = kisi.get().getPublicKey();
    if (publicKeyBase64 == null || publicKeyBase64.isBlank()) {
        throw new IllegalStateException("missing public key");
    }

    byte[] keyBytes = Base64.getDecoder().decode(publicKeyBase64.replaceAll( regex: "\\s+",  replacement:
    X509EncodedKeySpec spec = new X509EncodedKeySpec(keyBytes);
    KeyFactory kf = KeyFactory.getInstance( algorithm: "RSA");
    PublicKey publicKey = kf.generatePublic(spec);

    Cipher cipher = Cipher.getInstance( transformation: "RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
    cipher.init(Cipher.ENCRYPT_MODE, publicKey);
    byte[] encrypted = cipher.doFinal(jsonPayload.getBytes(StandardCharsets.UTF_8));
    return Base64.getEncoder().encodeToString(encrypted);
}
```

3.1.1. Kimlik Doğrulama ve Yetkilendirme

Transfer requests are: encrypted using **RSA (OAEP SHA-256) and** signed using **HMAC-SHA256**

Users are stored in the **Kisi** table and their permissions determine which operations they can perform. Permissions control access to: public/private pull, push and manage operations



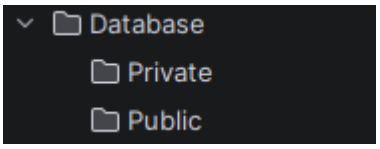
The server returns the following response codes0:

- 0 – Success (sometimes returned as 0|json)
- 1 – Signature/HMAC or payload format error
- 2 – RSA decryption, JSON parsing, or general format error
- 3 – User not found
- 4 – Incorrect password
- 5 – Resource/version not found

Permission levels are defined as:

- 0: "Public Puller"
- 1: "Public Puller + Pusher"
- 2: "Public Admin"
- 3: "Private Puller"
- 4: "Private Puller + Pusher"
- 5: "Private Admin"
- 6: "Public Admin + Private Puller"
- 7: "Public Admin + Private Puller + Pusher"
- 8: "Private Admin + Public Puller"
- 9: "Private Admin + Public Puller + Pusher"
- 10: "Public Puller + Private Puller"
- 11: "Private Puller + Pusher + Public Puller"
- 12: "Private Puller + Public Puller + Pusher"
- 13: "Public Puller + Pusher + Private Puller + Pusher"
- 14: "Global Admin"

3.1.2. Kaynak Yönetimi ve Versiyonlama



Public and private resources are stored on the filesystem. The **ResourceVersionSyncService** scans directories at application startup and synchronizes them with the database. The **ResourceVersion** table stores: scope, folder, version, uploader

```
Set<String> incomingKeys = collected.stream() Stream<ResourceVersion>
        .map(ResourceVersionSyncService::keyOf) Stream<String>
        .collect(Collectors.toSet());

List<ResourceVersion> existing = repo.findAll();
Set<String> existingKeys = existing.stream() Stream<ResourceVersion>
        .map(ResourceVersionSyncService::keyOf) Stream<String>
        .collect(Collectors.toSet());

Set<String> toDelete = new HashSet<>(existingKeys);
toDelete.removeAll(incomingKeys);

if (!toDelete.isEmpty()) {
    List<ResourceVersion> deleteEntities = existing.stream()
            .filter( ResourceVersion rv -> toDelete.contains(keyOf(rv)))
            .collect(Collectors.toList());
    repo.deleteAll(deleteEntities);
}

List<ResourceVersion> toInsert = collected.stream()
        .filter( ResourceVersion rv -> !existingKeys.contains(keyOf(rv)))
        .collect(Collectors.toList());
```

3.1.3. Pull (Download) Flow

The endpoint /api/private/transfer handles requests for: login, connect, check, pull. For pull operations the server generates: an AES key and chunk metadata which includes: chunk size, total chunks and file size.

```
try {
    Map<String, Object> obj = Map.of(
            k1: "response", v1: "0",
            k2: "aes_key", Base64.getEncoder().encodeToString(aesKey),
            k3: "chunk_size", chunkSize,
            k4: "total_chunks", totalChunks,
            k5: "file_size", fileSize
    );
    String json = objectMapper.writeValueAsString(obj);
    return send_file_json_internal(req.name, json);
} catch (Exception ex) {
    return "2";
}
```

The endpoint /api/private/transfer/zipchunk provides **chunked AES-GCM encrypted downloads**.

```
byte[] keyBytes = Base64.getDecoder().decode(publicKeyBase64.replaceAll( regex: "\\s+", replacement: ""));
X509EncodedKeySpec spec = new X509EncodedKeySpec(keyBytes);
KeyFactory kf = KeyFactory.getInstance( algorithm: "RSA");
PublicKey publicKey = kf.generatePublic(spec);

Cipher cipher = Cipher.getInstance( transformation: "RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
cipher.init(Cipher.ENCRYPT_MODE, publicKey);
byte[] encrypted = cipher.doFinal(jsonPayload.getBytes(StandardCharsets.UTF_8));
return Base64.getEncoder().encodeToString(encrypted);
```

3.1.4. Push (Upload) Flow

A push request opens a new upload session. The endpoint /api/private/transfer/pushzip receives **chunked AES-GCM encrypted uploads**.

```
byte[] aesKey;
try {
    aesKey = Base64.getDecoder().decode(aesKeyRaw);
} catch (IllegalArgumentException ex) {
    aesKey = aesKeyRaw.getBytes(StandardCharsets.UTF_8);
}

// start a new push session (single uploader assumption)
try {
    Path tempDir = Files.createTempDirectory( prefix: "malzahar_push_");
    Path tempZip = tempDir.resolve( other: "upload.zip");
    lastPush.set(new PushState(scope, klasor, aesKey, req.name, tempDir, tempZip));
    return "0";
} catch (Exception ex) {
    log.warn("handle_push: cannot create temp dir: {}", ex.toString());
    return "2";
}
```

When the final chunk arrives:

- the ZIP file is moved into the target directory
- a new version entry is written to the database

```
lastPush.set(new PushState(scope, klasor, aesKey, req.name, tempDir, tempZip));
```

3.1.5. Manage (Delete)

Specific versions can be deleted by administrators. Both the file and the corresponding **ResourceVersion** database record are removed from the database. Clients usually call **check** before performing a manage operation.

```
List<ResourceVersion> matches =
        resourceVersionRepository.findAllByScopeIgnoreCaseAndKlasorAndVersiyon(scope, klasor, versiyon);
if (matches.isEmpty()) {
    return "5";
}

ResourceVersion rv = matches.get(0);
String fileName = rv.getVersiyon() + " _ " + rv.getYukleyen() + ".zip";
String basePath = "Public".equalsIgnoreCase(scope) ? publicPath : privatePath;
Path target = Path.of(basePath, klasor, fileName);
try {
    Files.deleteIfExists(target);
} catch (Exception ex) {
    return "2";
}
```

3.2. Web Site

The web site portion is used as my **personal portfolio**. When users visit the website they interact only with this section.

3.2.1. Static Content Serving

The **HomeController** returns the homepage. Using **WebConfig**, resources such as: /, /Photos, /PDFs are served. Reports are available in **both Turkish and English versions**.

```
@Controller   Famura71
public class HomeController {
    @GetMapping(value = "/", produces = MediaType.TEXT_HTML_VALUE)   Famura71
    public ResponseEntity<Resource> index() {
        Resource page = new ClassPathResource("Server/index.html");
        return ResponseEntity.ok(page);
    }
}
```

```
public class WebConfig implements WebMvcConfigurer {
    @Override   no usages   Famura71
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler( ...pathPatterns: "/**")
                .addResourceLocations("classpath:/Server/");
        registry.addResourceHandler( ...pathPatterns: "/Photos/**")
                .addResourceLocations("classpath:/Photos/");
        registry.addResourceHandler( ...pathPatterns: "/PDFs/**")
                .addResourceLocations(
                        "file:src/main/resources/PDFs/",
                        "classpath:/PDFs/"
                );
    }
}
```

3.2.2. Interface

The website follows a **single-page portfolio design**.Language switching (TR / EN) is implemented using a **JavaScript dictionary system**. The interface includes sections for: projects, experience, references, contact. The reports page also supports: embedded PDF preview, opening reports in a new browser tab.

```
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Furkan Ahmet Karabulut</title>
<link rel="stylesheet" href="/styles.css">
<link rel="icon" type="image/png" href="/Photos/Nexus.png">
<script defer src="/app.js?v=2026-02-05"></script>
```

```
  "references.two.note": "Kendisi ile iş arama sürecimde tanıştım. ISSD adlı şirketin kurucusu v
  "contact.title": "İletisim",
  "contact.nameLabel": "İsim",
  "contact.namePlaceholder": "İsim",
  "contact.messageLabel": "Mesaj",
  "contact.messagePlaceholder": "Mesaj",
  "contact.fileLabel": "Dosya Sec",
  "contact.send": "Gönder",
  "footer.copyright": "© 2026 Furkan Ahmet Karabulut. Bütün hakları saklıdır"
},
en: {
  "nav.about": "About",
  "nav.projects": "Projects",
  "nav.experience": "Experience",
  "nav.contact": "Contact",
  "nav.references": "References",
  "hero.eyebrow": "Personal Portfolio",
  "hero.title": "Hi, I'm Furkan Ahmet Karabulut.",
  "hero.lead": "Junior Fullstack Software Engineer",
  "hero.ctaProjects": "View Projects",
  "hero.ctaContact": "Contact Me",
```

```
        <li>
          <a href="/PDFs/EN/White/Resume.pdf" data-pdf="/PDFs/EN/White/Resume.pdf">Resume</a>
          <a href="/PDFs/EN/White/Resume.pdf" target="_blank" rel="noopener">Open in a New Tab</a>
        </li>
      </ul>
    </div>
    <div class="reports-group">
      <h3>Orange</h3>
      <ul>
        <li>
          <a href="/PDFs/EN/Orange/Janus.pdf" data-pdf="/PDFs/EN/Orange/Janus.pdf">Janus</a>
          <a href="/PDFs/EN/Orange/Janus.pdf" target="_blank" rel="noopener">Open in a New Tab</a>
        </li>
        <li>
          <a href="/PDFs/EN/Orange/Theia.pdf" data-pdf="/PDFs/EN/Orange/Theia.pdf">Theia</a>
          <a href="/PDFs/EN/Orange/Theia.pdf" target="_blank" rel="noopener">Open in a New Tab</a>
        </li>
        <li>
          <a href="/PDFs/EN/Orange/Malzahar.pdf" data-pdf="/PDFs/EN/Orange/Malzahar.pdf">Malzahar</a>
          <a href="/PDFs/EN/Orange/Malzahar.pdf" target="_blank" rel="noopener">Open in a New Tab</a>
        </li>
```

3.3. Contact Form

The endpoint /api/contact endpoint'i accepts multipart form submissions. Uploaded files are processed as follows: File extensions are validated using a whitelist / Files are stored temporarily. / Files are scanned using **ClamAV**. / Clean files are sent via **SMTP using JavaMail**.

Sensitive keys (SMTP credentials, RSA keys, HMAC keys) are stored in application.properties. These keys were intentionally removed from the report and build files.

```
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=kate.famura@gmail.com
spring.mail.password=
```

```
if (contactForm) {
  contactForm.addEventListener( type: "submit",  listener: async (event : Event ) : Promise<void>  => {
    event.preventDefault();
    const submitButton : Element  = contactForm.querySelector( selectors: "button[type=\"submit\"]");
    if (submitButton) {
      submitButton.disabled = true;
      submitButton.textContent = currentLanguage === "en" ? "Sending..." : "Gonderiliyor...";
    }
    if (contactStatus) {
      contactStatus.textContent = "";
    }
    try {
      const formData : FormData  = new FormData(contactForm);
      const response : Response  = await fetch( input: contactForm.getAttribute( qualifiedName: "action") || "/api/contact",  init: {
        method: "POST",
        body: formData
      });
      if (response.ok) {
        if (contactStatus) {
          contactStatus.textContent = currentLanguage === "en" ? "Sent" : "Gönderildi";
        }
        contactForm.reset();
      } else {
        if (contactStatus) {
          contactStatus.textContent = currentLanguage === "en" ? "An error occurred" : "Bir hata ile karşılaşıldı";
        }
      }
    } catch (error) {
      if (contactStatus) {
        contactStatus.textContent = currentLanguage === "en" ? "An error occurred" : "Bir hata ile karşılaşıldı";
      }
    } finally {
      if (submitButton) {
        submitButton.disabled = false;
        submitButton.textContent = currentLanguage === "en" ? "Send" : "Gonder";
      }
    }
```

4. Project Analysis

4.1. Test

The project has been tested on Windows and Linux. It had been tested with remote connections from other cities, remote connections from other countries. All tests passed successfully and the website operates correctly. Both the **transfer subsystem** and the **website subsystem** were tested.

4.2. Upgradeable Things

Adding or removing projects on the website is currently handled by static HTML and JavaScript. Unlike the transfer subsystem, this part is **not database-driven**. This could be improved by storing photos, PDF files and project metadata inside the SQLite database as well.

5. Project Comments

5.1. Possible Use Cases

- Use together with Malzahar
- Personal portfolio and self-presentation
- Demonstrating example projects during job applications

5.2. To-Do's When Using

Running the project inside **IntelliJ IDEA** is recommended.

This makes it easier to:

- observe runtime errors
- access the terminal
- debug the system

5.3. Developer Comments

Açıkcası At first, I planned to move the project to a dedicated web server in the future since I am currently using tunneling due to budget limitations.

However, after integrating the transfer system, it seems that **Kate (my Ubuntu MATE Linux machine)** has become the permanent host of this project.

-Hooah,
Famura

6. Credits

- Famura Only Project

7. References

- AES: National Institute of Standards and Technology. (2001), Advanced Encryption Standard (AES) (FIPS PUB 197) https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf
- AES-GCM: Dworkin, M. (2015). Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf
- RSA: Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM, 21(2), 120–126. https://doi.org/10.1145/359340.359342
- HTTPS: Rescorla, E. (2000). HTTP over TLS. Internet Engineering Task Force (RFC 2818). https://datatracker.ietf.org/doc/html/rfc2818
- WinHTTP: Microsoft. (2024). WinHTTP API documentation. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/winhttp/about-winhttp
- Cryptography API: Microsoft. (2024). Cryptography API: Next Generation (CNG). Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/seccng/cng-portal
- OpenSSL: OpenSSL Project. (2024). OpenSSL: Cryptography and SSL/TLS Toolkit. https://www.openssl.org/docs/
- PowerShell ZIP: Microsoft. (2024). Compress-Archive. Microsoft Learn. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.archive/compress-archive

8. Disk Contents

- Source code of the project
- This report in PDF format and Turkish version
- This report in DOCX format and Turkish version
- Tell Me True by Sarah Jarosz